Mothercat Organization API: Frontend Integration Guide

This guide covers the staff directory, CSV import, and department operations available to the frontend. The machine-readable source of truth is `../contracts/openapi/organization.yaml`.

Scope and endpoint summary

The Organization API has 10 blueprint operations:

1. `GET /api/v1/staff` — search and paginate staff (`staff.read`).
2. `POST /api/v1/staff` — create a pending staff profile (`staff.create`).
3. `GET /api/v1/staff/{staffId}` — get a staff profile (`staff.read`).
4. `PATCH /api/v1/staff/{staffId}` — update profile fields (`staff.update`).
5. `POST /api/v1/staff/{staffId}/deactivate` — deactivate employment (`staff.deactivate`).
6. `POST /api/v1/staff/imports` — validate and import a CSV (`staff.import`).
7. `GET /api/v1/staff/imports/{importId}` — read an import result (`staff.import`).
8. `GET /api/v1/departments` — list departments (`staff.read`).
9. `POST /api/v1/departments` — create a department (`departments.manage`).
10. `PATCH /api/v1/departments/{departmentId}` — update a department (`departments.manage`).

The pre-existing `POST /api/v1/staff/{staffID}/provision` account-onboarding workflow is documented separately below. It is not one of those 10 blueprint operations.

Identifiers: display number versus API resource ID

A staff profile has two different identifiers:

- `staff_id` is the organization's official staff number. It is exactly four or five digits, for example `"23847"`.
- `id` is the API's internal UUID for that staff profile. Use this UUID in `{staffId}` path segments for get, update, deactivate, and provisioning.

The frontend may display the official number as **ID:23847**, but the `ID:` prefix is presentation only. Send and store `"23847"` as the `staff_id` value; do not include the prefix in request bodies, CSV values, login, or search.

For example, given this API response:

```
{

  "staff": {
```

```
    "id": "7da27cda-3496-4d02-9d99-fb0e956297e5",
    "staff_id": "23847",
    "full_name": "Ada Okafor",
    "employment_status": "pending"
  }
}
```

Render the staff number as `ID:23847`, while using `7da27cda-3496-4d02-9d99-fb0e956297e5` in a profile URL. Do not construct a profile URL using the official staff number.

Department `id`, staff `id`, `account_id`, and import `id` values are UUIDs. `account_id` can be `null` until a login account is associated.

Environments and base URL

Use the API origin supplied by the backend/deployment owner for each environment. Do not hard-code a production hostname. For example:

```
VITE_API_BASE_URL=https://<api-host>
```

The base URL is the API origin, without `/api/v1`. The paths in this guide include `/api/v1`. Deployed requests must use HTTPS. Never put database connection strings, migration credentials, service tokens, or passwords in frontend configuration.

Authentication, permissions, and browser behavior

- Every Organization endpoint requires a valid Identity session cookie named `mothercat_session`.
- For browser `fetch`, include `credentials: "include"` so the browser sends the HttpOnly cookie. Do not attempt to read or set the cookie in JavaScript.
- The API checks the permission assigned to the authenticated user. Hiding a button in the frontend is a user-experience choice, not authorization; the API remains authoritative and may return `403 permission_denied`.
- All mutation operations require an `Origin` matching the API request's scheme and host. Browsers supply this header; frontend JavaScript cannot set it manually. A cross-origin frontend cannot call mutations successfully merely because CORS allows the frontend origin. Coordinate hosting/origin policy with the backend owner.
- For credentialed cross-origin requests, the API would need to allow the exact frontend origin and credentials; wildcard `Access-Control-Allow-Origin` is not valid with credentials. The current mutation Origin check is stricter than CORS.
- Browser cookie behavior also depends on the deployed frontend/API site relationship. Confirm that the frontend and API hosting domains work with the current Secure, HttpOnly, SameSite cookie settings before integration.

Common browser request setup:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;



const response = await fetch(`${API_BASE_URL}/api/v1/staff`, {

  credentials: "include",

  headers: { Accept: "application/json" },

});
```

For JSON mutations, add `Content-Type: application/json` and `Origin` is handled by the browser. Do not manually set `Origin`.

Common request and error conventions

JSON request bodies require:

```
Content-Type: application/json

Accept: application/json
```

Send exactly one JSON object. Unknown JSON fields are rejected. The server limits JSON request bodies to 16 KiB.

Errors use this envelope:

```
{

  "error": {

    "code": "invalid_request",

    "request_id": "request-correlation-id"

  }

}
```

Some endpoints also include a safe `message`. Use `error.code` to select frontend behavior and retain `request_id` for support. Do not show stack traces or log session cookies, credentials, or full private records.

Common status meanings:

- `400`: invalid input, query, UUID, or CSV request.
- `401`: missing, expired, or invalid session.
- `403`: missing permission or mutation Origin not accepted.
- `404`: referenced staff, department, or import was not found.
- `409`: duplicate official staff number or department name.

- `413`: import upload exceeded the request-size limit.
- `415`: unsupported content type.
- `422`: CSV rows failed validation; no staff rows from that batch were imported.
- `500`: unexpected server failure.
- `503`: Identity, database, audit, or another required dependency is unavailable.

Staff directory

1. List, search, and paginate staff

`GET /api/v1/staff`

Requires `staff.read`. Supported query parameters:

Parameter	Type and bounds	Meaning
`limit`	Integer, 1–100; default 50	Maximum records in the page
`offset`	Integer, 0–1,000,000; default 0	Number of matching records to skip
`search`	String, at most 100 characters	Case-insensitive substring match in official staff number, full name, or work email
`department_id`	Department UUID	Restrict to a department
`employment_status`	`pending`, `active`, `inactive`, or `deactivated`	Restrict by employment status

Only these query parameters are accepted, and each may appear only once.

Example:

```
GET /api/v1/staff?limit=25&offset=0&search=23847&employment_status=active

Accept: application/json

Cookie: mothercat_session=<browser-managed>
```

Success: `200 OK`.

```
{

  "staff": [

    {

      "id": "7da27cda-3496-4d02-9d99-fb0e956297e5",

      "account_id": null,
```

```
      "staff_id": "23847",
      "department_id": "1c9491b8-21ce-4c66-a025-006bb346b810",
      "full_name": "Ada Okafor",
      "work_email": "ada@example.org",
      "job_title": "Site Engineer",
      "employment_status": "active"
    }
  ],
  "pagination": {
    "limit": 25,
    "offset": 0,
    "total": 1
  }
}
```

`total` is the number of records matching the filters, not only the number returned on the current page. An empty result is a successful response with `"staff": []`.

Possible errors: `400 invalid_query`, `401 unauthorized`, `403 permission_denied`, `500 internal_error`, `503 service_unavailable` or `identity_unavailable`.

2. Create a staff profile

`POST /api/v1/staff`

Requires `staff.create`, a session, and an accepted matching Origin. This creates a staff profile only; it does not create login credentials or assign a role.

Request:

```
{
  "staff_id": "23847",
  "full_name": "Ada Okafor",
  "work_email": "ada@example.org",
  "job_title": "Site Engineer",
  "department_id": "1c9491b8-21ce-4c66-a025-006bb346b810"
```

```
}
```

`staff_id` and `full_name` are required. The official number must contain exactly four or five digits. `work_email`, `job_title`, and `department_id` are optional and may be `null`. A department UUID must refer to an existing department. New profiles start with `employment_status: "pending"`.

Success: `201 Created`, with `{ "staff": { ... } }`.

Possible errors: `400 invalid_request` or `invalid_staff_record`, `401`, `403`, `409 duplicate_staff_id`, `415`, `500`, and `503`. A nonexistent department returns a department-not-found error.

3. Get a staff profile

`GET /api/v1/staff/{staffId}`

Requires `staff.read`. Replace `{staffId}` with the staff profile's internal UUID from the `id` field—not the official four/five-digit `staff_id`.

Success: `200 OK`, with `{ "staff": { ... } }`.

Possible errors: `400 invalid_staff_id` for a malformed UUID, `401`, `403`, `404 staff_not_found`, `500`, and `503`.

4. Update a staff profile

`PATCH /api/v1/staff/{staffId}`

Requires `staff.update`, a session, and a matching Origin. The path uses the internal staff UUID. Editable fields are `full_name`, `work_email`, `job_title`, and `department_id`.

Only send fields to change. Omit unchanged fields. Send JSON `null` for nullable fields to clear them. The official `staff_id`, account association, and employment status are not editable through this endpoint.

Example to update the title and clear the email:

```
{

  "job_title": "Senior Site Engineer",

  "work_email": null

}
```

Success: `200 OK`, with `{ "staff": { ...updated profile... } }`.

Possible errors: `400 invalid_request`, `invalid_staff_record`, or `department_not_found`; `401`, `403`, `404 staff_not_found`, `415`, `500`, and `503 audit_unavailable` or `service_unavailable`.

5. Deactivate staff employment

`POST /api/v1/staff/{staffId}/deactivate`

Requires `staff.deactivate`, a session, and a matching Origin. The path uses the internal staff UUID. There is no request body.

For a staff profile with a linked Identity account, the backend first suspends the account, revokes sessions, and invalidates outstanding recovery/activation tokens. It then marks employment inactive. If Identity suspension fails, the employment update is not performed. If suspension succeeds but the later Organization update fails, the account remains suspended and the failure must be resolved by an administrator. The last active SuperAdmin cannot be suspended.

Success: `200 OK`, with `{ "staff": { ...employment_status: "inactive"... } }`. This endpoint does not reactivate employment.

Possible errors: `400 invalid_staff_id`, `401`, `403`, `404 staff_not_found`, `500`, and `503` if audit, Identity, or Organization dependencies are unavailable.

Staff CSV import

6. Upload a staff CSV

`POST /api/v1/staff/imports`

Requires `staff.import`, a session, and a matching Origin. Upload one file as multipart field `file`; do not send JSON. The HTTP client must generate the multipart `Content-Type` including its boundary. In browser `fetch`, do not set `Content-Type` manually when sending `FormData`.

CSV requirements:

- Required headers: `staff_id`, `full_name`.
- Optional headers: `work_email`, `job_title`, `department_id`.
- The official staff number in `staff_id` must be exactly four or five digits, without the display-only `ID:` prefix.
- Header order is unrestricted. Unknown or duplicate headers fail validation.
- Maximum request/file size: 5 MiB. Maximum data rows: 5,000.
- Blank rows are ignored. Duplicate staff numbers in the file, existing numbers, invalid fields, or nonexistent departments cause the whole batch to fail.
- The batch is all-or-nothing. A successful import creates pending staff profiles only; it does not create login accounts or assign roles.
- Processing is synchronous. Failed results retain up to 100 row issues and indicate if the error list was truncated.

Example CSV:

```
staff_id,full_name,work_email,job_title,department_id

23847,Ada Okafor,ada@example.org,Site Engineer,1c9491b8-21ce-4c66-a025-006bb346b810

1234,Sam Mensah,sam@example.org,Technician,
```

Browser example:

```
const form = new FormData();

form.append("file", csvFile);



const response = await fetch(`${API_BASE_URL}/api/v1/staff/imports`, {

  method: "POST",

  credentials: "include",

  headers: { Accept: "application/json" },

  body: form,

});
```

Success: `201 Created`.

```
{

  "import": {

    "id": "7da27cda-3496-4d02-9d99-fb0e956297e5",

    "status": "completed",

    "created_at": "2026-11-02T18:00:00Z",

    "completed_at": "2026-11-02T18:00:01Z",

    "result": {

      "total_rows": 2,

      "imported_rows": 2,

      "error_count": 0

    }

  }

}
```

Validation failure: `422 Unprocessable Entity`. No staff rows from that batch are committed. The response includes a failed import result:

```
{

  "error": {

    "code": "staff_import_validation_failed",
```

```
    "request_id": "request-correlation-id"
  },
  "import": {
    "id": "7da27cda-3496-4d02-9d99-fb0e956297e5",
    "status": "failed",
    "created_at": "2026-11-02T18:00:00Z",
    "completed_at": "2026-11-02T18:00:00Z",
    "result": {
      "total_rows": 2,
      "imported_rows": 0,
      "error_count": 1,
      "errors": [
        {
          "row": 3,
          "staff_id": "123",
          "code": "invalid_staff_record",
          "message": "staff ID, name, or optional field is invalid"
        }
      ]
    }
  }
}
```

Row numbers in issues refer to CSV line numbers, including the header line. Present the returned row issue messages to the user in a safe, actionable way. Other errors include `400 invalid_import_upload` or `file_field_required`, `401`, `403`, `413 import_file_too_large`, `415 content_type_must_be_multipart_form_data`, `500 staff_import_failed`, and `503`.

7. Get an import result

`GET /api/v1/staff/imports/{importId}`

Requires `staff.import`. Use the import UUID returned by the upload response. Success: `200 OK`, with `{ "import": { ... } }`.

Possible errors: `400 invalid_import_id`, `401`, `403`, `404 staff_import_not_found`, `500`, and `503`.

Departments

Department UUIDs are used for department resource paths and staff `department_id` values.

8. List departments

`GET /api/v1/departments`

Requires `staff.read`. No query parameters or request body.

Success: `200 OK`:

```
{
  "departments": [
    {
      "id": "1c9491b8-21ce-4c66-a025-006bb346b810",
      "name": "Engineering",
      "description": "Engineering and site operations"
    }
  ]
}
```

Possible errors: `401`, `403`, `500`, and `503`.

9. Create a department

`POST /api/v1/departments`

Requires `departments.manage`, a session, and matching Origin.

Request:

```
{
  "name": "Engineering",
  "description": "Engineering and site operations"
}
```

`name` is required. `description` is optional or `null`. Success: `201 Created`, with `{ "department": { ... } }`.

Possible errors: `400 invalid_request` or `invalid_department`, `401`, `403`, `409 department_name_conflict`, `415`, `500`, and `503 audit_unavailable` or `service_unavailable`.

10. Update a department

`PATCH /api/v1/departments/{departmentId}`

Requires `departments.manage`, a session, and matching Origin. The path uses the department UUID. Send only fields to change; `name` cannot be empty or cleared. Send `description: null` to clear the description.

Example:

```
{ "description": null }
```

Success: `200 OK`, with `{ "department": { ...updated department... } }`.

Possible errors: `400 invalid_department_id`, `invalid_request`, or `invalid_department`; `401`, `403`, `404 department_not_found`, `409 department_name_conflict`, `415`, `500`, and `503`.

Existing supplementary account provisioning

`POST /api/v1/staff/{staffID}/provision`

This existing onboarding workflow is additional to the 10 operations above. Despite the placeholder name, `{staffID}` here is also the internal UUID of the Organization staff profile. It is not the official numeric staff number.

Requires `user_roles.assign`, a session, and matching Origin. It provisions an inactive login account for an eligible pending staff profile, assigns the permitted staff role, and sends activation instructions to the work email. The profile must have a work email and must not already be provisioned.

Success: `200 OK` with `staff` and `invitation_sent: true`. Possible errors include `400`, `401`, `403`, `404`, `409`, and `503`. The user activates the account using the separate Identity activation workflow.

Suggested frontend integration sequence

1. Restore the signed-in user from the Identity session endpoint and retain the server-returned permissions for UI presentation.
2. Load departments and the first staff page in parallel, handling `401` as a signed-out session and `403` as insufficient permission.
3. Use the staff `id` UUID for profile, update, deactivate, and provisioning requests. Display `staff_id` as `ID:<staff_id>` only in the UI.
4. After create, update, deactivate, or a successful import, refresh the affected list/detail data from the API.
5. For imports, show progress while the synchronous request is running, then display the batch totals and row issues returned by the API.
6. Preserve `request_id` when showing or logging a request failure, but never log cookies, passwords, activation/reset tokens, or confidential record contents.

Additional contract

Refer to `../contracts/openapi/organization.yaml` for the complete OpenAPI schemas, query parameter definitions, security requirements, and documented response codes.